



PredTrAD – Prediction-based Transformer for Anomaly Detection in Multivariate Time Series Data

Jan Schuster^{1,2}, Alexander Wölfel², Fabian Brunner¹, Christian Bergler¹

¹Department of Electrical Engineering, Media and Computer Science, Technical University of Applied Sciences Amberg-Weiden, Germany

²Technological Institute for Applied Artificial Intelligence GmbH, Weiden, Germany

j.schuster@oth-aw.de, c.bergler@oth-aw.de

Abstract

Anomaly detection in multivariate time series data is an extensive field of research with significant impact on a broad spectrum of real-world applications. Building a reliable anomaly detection system is extremely challenging due to data imbalance, lack of labels, next to the actual definition of anomalies. The current study proposes a new transformer-based approach, together with a publicly available time series dataset – the TIKI data corpus – comprising various performance metrics of a Kubernetes cluster distributed over roughly 500 million timestamps. The proposed network, entitled PredTrAD, is compared with the state-of-the-art anomaly detection models – TranAD and DTAAD – in several experiments: (1) verification on three well-known benchmark corpora, (2) evaluation on the TIKI data, and (3) cross-entity anomaly detection. Across all experimental constellations, the proposed model has shown a significant performance growth, proven by F1 score enhancements of 2.41% up to 16.81%.

Index Terms: Anomaly Detection, Deep Learning, Transformer, Unsupervised Learning

1. Introduction

The rapid and global process of digitization has led to huge amount of time series data which require monitoring and downstream analysis. Anomaly detection, defined as the process of identifying data points that significantly deviate from the majority of instances [1], has become increasingly important in this context. Deep learning methods have emerged as highly promising in this research field, offering the ability to learn complex patterns directly from raw data without extensive feature engineering [2]. However, anomaly detection for time series data faces several challenges: patterns are highly domain-specific [3], labeling requires significant expertise, and class imbalance can lead to accuracy paradox issues [4]. Additionally, practical systems must carefully balance sensitivity with precision to avoid alert fatigue [4]. The current study proposes leveraging the transformer architecture [5] for time series anomaly detection, introducing a novel model architecture: the Prediction-based Transformer for Anomaly Detection (PredTrAD), in two different versions V1 and V2 (see Figure 3). These architectures combine parallel sequence processing with positional encoding and self-attention mechanisms, enabling efficient processing of large-scale temporal data [6, 7]. The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 describes the data material. Section 4 presents the proposed methodology, followed by our experimental setup, results and a discussion in Section 5. Finally, a summary, conclusion, and future outlook are given in Section 6.

2. Related Work

Recent advances in deep learning have transformed anomaly detection, particularly through the transformer-based architectures [5]. The current state-of-the-art methods are TranAD [7] and DTAAD [6]. TranAD uses an encoder and two decoder networks, implementing a two-phase adversarial training process with model-agnostic meta learning (MAML) [7]. It calculates a focus score to evaluate reconstruction quality and uses the Peak Over Threshold (POT) procedure [8] to identify anomalies. Building on TranAD, DTAAD combines temporal convolutional networks (TCN) with the transformer architecture [6]. It consists of two main components: a TCN-based autoregression layer and a Transformer encoder-decoder layer. Like TranAD, it uses two-phase adversarial training and MAML.

3. Data Material

In this study three commonly used benchmark datasets are used, all of them widely applied in the research field of anomaly detection to be able to properly evaluate and compare the quality of different anomaly detection approaches. The two benchmark repositories – Soil Moisture Active Passive (SMAP) satellite data & Mars Science Laboratory (MSL) rover data – describe two public data corpora from NASA, both proposed by Hundman et al. [9]. They contain telemetry data like radiation, temperature, power, computational activities, et cetera. SMAP monitors 55 different entities across 25 dimensions, while MSL encompasses 27 entities with 55 monitored dimensions. The labels were derived from Incident Surprise Anomaly reports and therefore all telemetry channels were reviewed to ensure that the anomalies are in the corresponding telemetry data, and specific anomalous time ranges among all dimensions were labeled manually by experts [9]. The Server Machine Dataset (SMD) is similar to the novel and proposed TIKI dataset, as it also includes metrics like CPU load, network usage, memory consumption, et cetera [10]. The dataset consists of 28 entities, with each entity representing a server, monitoring 38 different metrics. In general, the SMD dataset was presented by Su et al. [10] and contains data collected over a five-week period from a large internet company. The test data was separately labeled by domain experts for each dimension of each entity based on incident reports. The industry partner TIKI has its own data platform, named TIKI Data Science Platform (DSP), which provides companies and their own data scientists a platform for machine learning operations (MLOps). Almost every component of the TIKI-DSP is part of a large Kubernetes cluster, which utilizes Container Advisor to gather information about different metrics (e.g. resource usage, open threads, etc.) of a specific container (similar to virtual machines), and stores it in a time series database. This data describes the proposed TIKI dataset, a time series corpus including 35,559 pods. A pod refers to a

collection of application containers, as a group and single deployable unit. Each pod consists of 29 selected metrics. Within each metric, there are (N) time series values, all corresponding to the same (N) timestamps. In total, the dataset comprises 491,861,286 entries for all pods, which sums up to a total time span of 170,785 days, based on a data resolution of one timestamp per 30 seconds. Table 1 illustrates all the utilized datasets, later used in different experimental setups (see Section 4).

4. Methodology

4.1. Multivariate time series

The one-dimensional time series $\mathcal{T}$, which is sequentially ordered by a set of timestamps $T = \{1, 2, 3, \dots, t\}$ can be written as

$$\mathcal{T} = \{x_1, x_2, \dots, x_t\}, \quad (1)$$

where the variable x_i represents the data point at timestamp $i \in T$. A multivariate time series X tracks multiple variables measured simultaneously over time and can be formulated as matrix $X \in \mathbb{R}^{t \times d}$

$$X = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{t1} & x_{t2} & \dots & x_{td} \end{pmatrix}, \quad (2)$$

where the i -th row of the matrix X refers to the data at timestamp $i \in \{1, 2, \dots, t\}$, and the j -th column represents the entire time series $\mathcal{T}_j$ of dimension $j \in \{1, 2, \dots, d\}$. For any new time series $\mathcal{T}$ with the same feature characteristics as the training data ($d_{\text{train}} = d_{\text{test}}$) of length $\hat{T}$, we need to predict $y = \{y_1, y_2, \dots, y_{\hat{T}}\}$, where $y_k \in \{0, 1\}$ denotes whether the data point at the t -th timestamp is anomalous or not.

4.2. Data Preprocessing

The raw multivariate time series data matrix X (see Equation 2), is not directly utilized for the anomaly detection procedure. The entire data corpora were normalized by applying a min-max or standardization (mean/stdv) scaling procedure in order to ensure consistency and comparability with preprocessing approaches deployed in previous research [6, 7]. Furthermore, the data matrix X is transformed into a list of sliding windows (see Figure 1) of fixed window size w . The resulting tensor is denoted as $\hat{X} \in \mathbb{R}^{t \times w \times d}$.

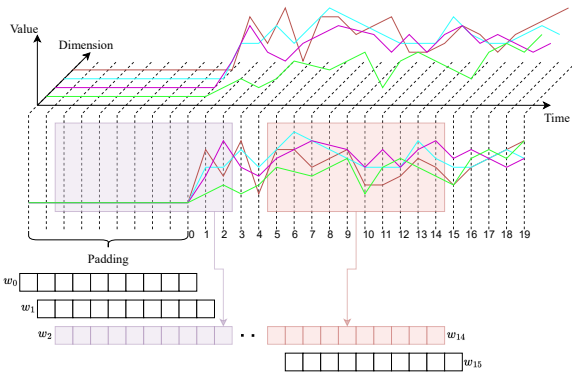


Figure 1: Illustration of the preprocessing steps of a multivariate time series data comprising four dimensions and 20 timestamps to transform it into a list of sliding windows $W \in \{w_0, w_1, \dots, w_{19}\}$ with a window size $w = 10$

4.3. Network Model

The proposed PredTrAD model is available in two different versions – PredTrAD-V1 and PredTrAD-V2 (see Figure 3). PredTrAD was fundamentally inspired by the transformer concept [5], originally designed for the task of natural language processing. Figure 3 provides a schematic representation of the general architectural design, including both model versions. In principle, these models are designed to operate prediction-based, as opposed to reconstruction-based methods, such as TranAD and DTAAD [6, 7]. To illustrate, Figure 2 shows the input and the desired output of PredTrAD. The window size w is defined by the two variables w_1 and w_2 , which are the length of src and tgt , respectively.

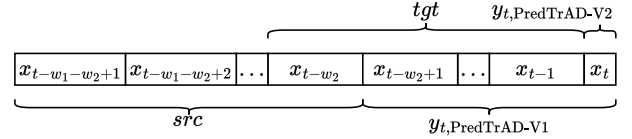


Figure 2: Illustration of a single window and the split into model inputs (src & tgt) and desired outputs of PredTrAD-V1 and PredTrAD-V2

Figure 2 illustrates, that src and tgt overlap at element x_{t-w_2} . This element serves as a start token, which means that PredTrAD-V1 is trained to predict the subsequent elements designated as $y_{t,PredTrAD-V1}$. For this reason, we utilize masked multi-head attention in the decoder to exclude future predictions and consider only previous elements of the respective window. PredTrAD-V2 is trained in a manner analogous to PredTrAD-V1, yet its fundamental objective is to predict solely the final element of the respective time window as illustrated in Figure 2. The left part of Figure 3 is similar to the original transformer architecture [5]. At the beginning, $src \in \mathbb{R}^{b \times w_1 \times d}$ and $tgt \in \mathbb{R}^{b \times w_2 \times d}$, where b refers to batch size and w_1 and w_2 denote the length of the src and tgt windows, respectively, are fed into a linear layer to transform the parametric dimensionality d to d_{model} , followed by a learnable positional encoding, which utilizes normal positional encoding in combination with a feed-forward network to add more flexibility to this fundamental process. The computational flow of the encoder-decoder architecture remains unchanged compared to the description provided by Vaswani et al. [5]. The outcome of the decoder exhibits the shape $\mathbb{R}^{b \times w_2 \times d_{\text{model}}}$. PredTrAD-V1 uses this abstract representation and directs it to a linear layer that transforms the last dimension from d_{model} to d_{test} . As a result, the desired output would be $y_{t,PredTrAD-V1}$, as visualized in Figure 2. Considering Figure 3, it is notable that PredTrAD-V2 is basically an extension of PredTrAD-V1. This is accomplished by applying the contracting path of the original U-Net architecture [11] to the result of the transformer decoder, with the intention of extracting specific temporary features from this complex representation. As visualized in Figure 3, the green-highlighted part corresponds to the PredTrAD-V2 component, where the window size of the decoder input w_2 is set to $w_2 = 10$, and the model dimension d_{model} is defined as $d_{\text{model}} = 512$. The main idea behind the more complex PredTrAD-V2 is to treat the outcome of the transformer decoder as feature maps with the shape $\mathbb{R}^{b \times c \times w_2 \times d_{\text{model}}}$, where the additional variable c refers to the number of channels. The initial configuration of the first feature map comprises a single channel. Subsequently, a contracting step containing two consecutive convolutional layers that process the input, followed by max pooling to reduce spatial dimensions, is applied. The respective kernel sizes of the con-

Experiment	Dataset	Split Entity	Train	Validation	Test
			Samples [#]	Samples [#]	Samples [#]
1	SMAP	P-1	2,872	-	8,505
1	MSL	C-1	2,158	-	2,264
1	SMD	Machine-1-1	28,478	-	28,478
2	TIKI	web-6495d5f5f9	563,295	138,515	140,826
3	SMD	Machine-1	194,369	118,498	75,876

Table 1: Overview of dataset configurations across three experiments, showing the data splits for training, validation, and testing.

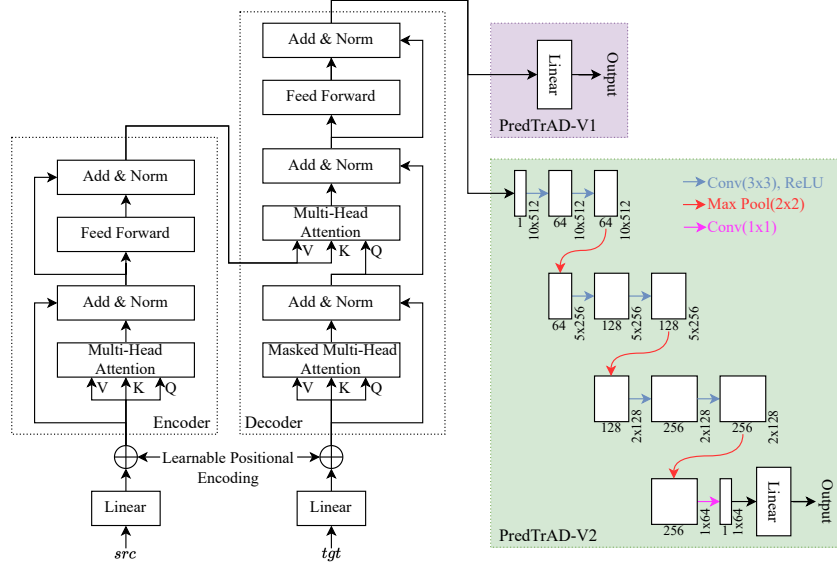


Figure 3: Comparison of the novel proposed PredTrAD-V1 and PredTrAD-V2 architectures

volution and max pooling procedure are described and highlighted in Figure 3. The PredTrAD-V2 model comprises three stages of contracting, where the number of channels increases from 1 to 256, next to a feature map dimensionality reduction from 10×512 to 1×64 . The number of channels in the final feature map decreases from 256 to 1 applying a 1×1 -convolution, resulting in an output dimension of $\mathbb{R}^{b \times 1 \times 1 \times 64}$. This output is then linearly transformed to align with the dimensions of the target prediction $y_{t, \text{PredTrAD-V2}} \in \mathbb{R}^{b \times 1 \times d_{\text{test}}}$. As a result, PredTrAD-V1 and PredTrAD-V2 are able to predict the last element of each sliding window by taking the previous timestamps into account, which results in a continuous prediction of the entire multivariate time series. Consequently, similarly to the TranAD and DTAAD approaches, this prediction is utilized to calculate anomaly scores, which are further used to determine an anomaly threshold that classifies each data point as either anomalous or not.

4.4. Anomaly Detection

During inference, the Mean Squared Error (MSE) between the prediction and the ground truth value is calculated for each timestamp and dimension. These values are then used as anomaly scores. As prior research has shown [6, 7, 9, 10, 12], for a fair comparison, we use the Peak Over Threshold (POT) method [8] that automatically determines the anomaly threshold based on the respective anomaly scores, which is further used to classify each timestamp as either anomalous or normal.

5. Experiments, Results, and Discussion

In this Section, we present three experiments and provide a brief overview of the purpose of the conducted analyses: (1) anomaly detection on benchmark data (see Section 5.1), (2) anomaly detection on the TIKI dataset (see Section 5.2), and (3) cross-entity anomaly detection (see Section 5.3). Subsequently, the anomaly detection results are presented and discussed, using plain confusion matrix metrics, together with the resulting False Positive Rate (FPR) and F1 score (F1). The best results (see Table 2) for anomaly detection are highlighted in bold.

5.1. Anomaly Detection – Baseline

The structure of this baseline experiment is conceivable straightforward. The experimental setup was kept equally to the previous research [6, 7], and the benchmark datasets (SMAP, MSL, SMD) were preprocessed as described in Section 4.2. Table 2 clearly illustrates that PredTrAD-V2 performs best in terms of FPR and FPs. It has the highest F1 score for the aggregated values, which is 2.41% higher than the second-best TranAD model.

5.2. Anomaly Detection on TIKI dataset

In general, this experiment is similar to the previous one, as the main objective is to evaluate the four anomaly detection models on a subset of the novel proposed TIKI dataset. The key distinction is the utilization of a validation dataset, which serves the purpose of validating the reconstruction/prediction performance of the respective method. The results of this experiment

Model & Data	SMAP, MSL, SMD					
	TP	TN	FP	FN	FPR [%]	F1 [%]
PredTrAD-V1	3,723	34,993	530	2	1.43	93.33
PredTrAD-V2	3,725	35,254	269	0	0.76	96.52
TranAD	3,725	35,057	466	0	1.31	94.11
DTAAD	3,725	34,693	830	0	2.34	89.98

Table 2: Performance across all three benchmark datasets (SMAP, MSL, SMD – see Section 3)

are displayed in Table 3. We observe that PredTrAD-V2 outperforms the models TranAD, DTAAD and PredTrAD-V1 with regard to the anomaly detection task. Additionally, it is noticeable that all models, except PredTrAD-V1, detect anomalies worse when the data are standardized.

5.3. Cross-Entity Anomaly Detection

Previous experiments examined only the entity-based anomaly detection that involves a single trained model for each entity. In this experiment, we evaluate the ability of a single model to detect anomalies across multiple entities. The core idea of this experiment arose due to the high number of entities (35,559 pods) of the TIKI dataset. The motivation behind this approach is that cross-entity anomaly detection would significantly reduce the maintenance effort associated with anomaly detection models for a dataset comprising a vast number of entities. Given the similarities between the TIKI and SMD dataset, particularly in their monitored metrics, 8 entities of the SMD dataset are employed in this experiment to explore cross-entity anomaly detection. The TIKI data corpus offers a comprehensive collection of metrics, however, it was not utilized due to the limited availability of anomaly labels, making the extensively labeled SMD dataset more suitable for evaluating anomaly detection performance. Two key methodological improvements have been implemented: First, we replace the original POT with Dynamic Streaming Peak Over Threshold (SPOT) [8] for threshold selection, as POT’s constant threshold based on both training and test anomaly scores proved inadequate. Dynamic SPOT instead provides an adaptive threshold derived only from test anomaly scores. Secondly, the validation procedure was enhanced by monitoring the models’ F1 scores on the validation dataset, directly evaluating their anomaly detection capabilities during the validation phase. Due to hardware constraints, only TranAD, DTAAD, and PredTrAD-V1 were optimized for the utilization of Dynamic SPOT. The complex architecture of PredTrAD-V2, combined with a lack of time and computational resources, made an efficient threshold optimization too complex for the time being. Table 4 presents the performance metrics across all

test entities. Considering Table 4, we observe that PredTrAD-V1 has by far the best cross-entity anomaly detection performance, as the F1 score is 15.13% higher compared to the best state-of-the-art method DTAAD. However, a distinct observation in this experiment is a noticeable increase regarding FNs, compared to the other experiments. The rise in FNs may stem from the dynamic adjustment of the anomaly threshold, set by the dynamic SPOT algorithm. This threshold adjustment, combined with the varied behavior of time series data across entities, may lead the methods to overlook certain anomalies, despite the fact of a correct model identification. While V2 produced accurate anomaly scores, which clearly identified the respective anomalies, its unoptimized combination with Dynamic SPOT led to inappropriate threshold adjustments, which not just explain the higher number of FNs, but also the obscuring regarding the actual high-quality performance of PredTrAD-V2.

Model	SMD - Machine-1					
	TP	TN	FP	FN	FPR [%]	F1 [%]
PredTrAD-V1	5,956	106,106	58	1,378	0.05	89.24
PredTrAD-V2	3,844	106,067	97	3,490	0.09	68.19
TranAD	4,641	105,344	820	2,693	0.77	72.43
DTAAD	4,637	105,622	542	2,697	0.51	74.11

Table 4: Performance across all test entities of SMD-Machine-1

6. Conclusion and Future Work

Two new transformer-based methods, PredTrAD-V1 and PredTrAD-V2, were developed for multivariate time series anomaly detection. Through three key experiments, their effectiveness was thoroughly evaluated. In the comparative analysis, PredTrAD-V2 demonstrated superior performance by outperforming existing methods, showing a 2.41% and 6.54% increase in F1 score compared to TranAD and DTAAD respectively, while significantly reducing the FPR to 0.76%. When tested on the TIKI dataset, PredTrAD-V2 with min-max scaling achieved an impressive F1 score of 95.50%, demonstrating a 4.42% improvement over DTAAD and 12.48% over TranAD. In the final experiment testing cross-entity anomaly detection, PredTrAD-V1 emerged as the top performer with 89.24% F1 score, surpassing TranAD by 16.81% and DTAAD by 15.13%. Due to its lack of optimization for Dynamic SPOT, PredTrAD-V2 exhibited elevated FN rates, while maintaining low false alarm rates during threshold adjustment procedures. This work opens up several promising avenues for future research. A more in-depth comparative analysis of prediction-based versus reconstruction-based strategies has the potential to provide valuable insights into the underlying causes of observed performance differences. Our hypothesis is that, given the original Transformer’s primary objective—to predict the next element $t + 1$ based on past values up to timestamp t —prediction-based methods may be more effective at capturing temporal dependencies and contextual signals, particularly in complex multivariate time series. Furthermore, the baseline experiment could be extended by investigating additional datasets across different domains to evaluate the anomaly detection performance in varied contexts. Moreover, optimizing the dynamic threshold may be a promising approach for the future. Furthermore, researchers may explore the dynamic SPOT approach in greater detail, particularly focusing on optimizing the threshold procedure that distinguishes between anomalous and normal data points. The entire source code, all relevant experiment scripts and a subset of the TIKI dataset are made publicly available on GitHub under BSD-3 license at <https://github.com/jan-176/PredTrAD>.

Scaler	Model						
		TP	TN	FP	FN	FPR [%]	F1 [%]
MinMax	PredTrAD-V1	934	137,086	494	1	0.36	79.05
	PredTrAD-V2	934	137,493	87	1	0.06	95.50
	TranAD	934	137,199	381	1	0.28	83.02
	DTAAD	934	137,398	182	1	0.13	91.08
Std	PredTrAD-V1	934	137,136	444	1	0.32	80.76
	PredTrAD-V2	934	137,476	104	1	0.08	94.68
	TranAD	934	137,139	441	1	0.32	80.87
	DTAAD	934	137,273	307	1	0.22	85.85

Table 3: Performance for the TIKI dataset

7. References

- [1] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM computing surveys (CSUR)*, vol. 54, no. 2, pp. 1–38, 2021.
- [2] G. Li and J. J. Jung, "Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges," *Information Fusion*, vol. 91, pp. 1–10, 2023.
- [3] N. Laptev, S. Amizadeh, and I. Flint, "Generic and scalable framework for automated time-series anomaly detection," in *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, 2015.
- [4] C. Freeman, J. Merriman, I. Beaver, and A. Mueen, "Experimental comparison and survey of twelve time series anomaly detection algorithms," *Journal of Artificial Intelligence Research*, vol. 72, 2021.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [6] L.-r. Yu, Q.-h. Lu, and Y. Xue, "Dtaad: Dual tcn-attention networks for anomaly detection in multivariate time series data," *Knowledge-Based Systems*, vol. 295, p. 111849, 2024.
- [7] S. Tuli, G. Casale, and N. R. Jennings, "Tranad: Deep transformer networks for anomaly detection in multivariate time series data," *arXiv preprint arXiv:2201.07284*, 2022.
- [8] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly detection in streams with extreme value theory," in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 1067–1075.
- [9] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 387–395.
- [10] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2828–2837.
- [11] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," 2015. [Online]. Available: <https://arxiv.org/abs/1505.04597>
- [12] P. Boniol, T. Palpanas, M. Meftah, and E. Remy, "Graphan: Graph-based subsequence anomaly detection," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 2941–2944, 2020.